

Práctica 5: Procesamiento del Lenguaje

UNR - TUIA - Procesamiento de Lenguaje Natural

Parte 0: Fuentes de Información de una Empresa

En la carpeta `fuentes_de_informacion`

(https://drive.google.com/drive/folders/1o59pNr_LytXY_qd4KkCh0UuvZ9hV_Evi?usp=sharing) se encontrarán todos los datos necesarios para el ejercicio.

Estructura de datos proporcionada

1. **Productos** (`productos.csv`) Archivo CSV con 20 productos de electrodomésticos.

- **Campos:** ID, Nombre, marca, categoría, precio, stock, garantía, especificaciones técnicas (potencia, capacidad, voltaje, peso), descripción.

Ejemplo:

```
P001,Licuadaora  
Roja,Cocina,TechHome,89.99,45,Rojo,1200,2.0L,220V,3.2,24,Licuadaora de  
alta potencia con 5 velocidades y jarra de vidrio
```

2. **Reseñas de Usuarios** (`resenas_usuarios/`) Carpeta con 85 archivos de texto (simulando mensajes de WhatsApp).

- **Formato:** Fecha, Usuario, Telefono, Comentario, Puntaje, Producto.
- **Características:** 85 usuarios, puntajes 1-5, sentimientos mixtos.

3. **Manuales de Productos** (`manuales_productos/`) Carpeta con 6 manuales técnicos en formato Markdown.

- **Contenido:** Especificaciones, componentes, procedimientos paso a paso (29 totales), relaciones (compatibilidad, componentes compartidos), solución de problemas.

Ejemplo de procedimiento:

```
#### PROCEDIMIENTO: Rallar Queso
```

1. Instalar disco rallador grueso en eje superior del bowl
2. Cerrar tapa de la procesadora
- ...

Parte 1: Transformación de los Datos (ETL)

En esta parte deberán procesar cada fuente de datos y almacenarla en el tipo de base de datos más apropiado.

Ejercicio 1.1. Reseñas → MongoDB (o JSON)

Objetivo: Extraer información estructurada de las reseñas en texto plano y guardarlas en formato JSON o MongoDB.

Pasos sugeridos:

1. Leer todos los archivos de texto de `resenas_usuarios/`.
2. Usar un LLM o extractor de datos estructurados para extraer:
 - Fecha, Usuario, Teléfono
 - Producto (nombre o ID)
 - Puntaje (1-5)
 - Texto del comentario
 - (*Opcional*) Sentimiento (positivo/neutral/negativo)
 - (*Opcional*) Aspectos mencionados (precio, calidad, etc.)
3. Guardar en un archivo `resenas.json` o insertar en una colección de MongoDB.

Formato JSON esperado:

```
{
  "id_resena": "resena_001",
  "fecha": "2024-10-15",
  "usuario": "Maria_Rodriguez",
  "telefono": "+54 9 11 2345-6789",
  "producto_id": "P001",
  "producto_nombre": "Licuadora Roja",
  "puntaje": 5,
  "comentario": "Hola! Quiero dejar mi opinión sobre la Licuadora Roja...",
  "sentimiento": "positivo",
  "aspectos_positivos": ["potente", "diseño"],
  "aspectos_negativos": ["ruidosa"]
}
```

1.2. Productos → SQL o Pandas DataFrame

Objetivo: Cargar el CSV de productos en una estructura que permita consultas SQL o filtrado con pandas.

Pasos sugeridos:

1. Leer el archivo `productos.csv`.
2. Opción A: Crear base de datos SQLite y tabla `productos`.
3. Opción B: Mantener como DataFrame de pandas.

Esquema SQL sugerido:

```
CREATE TABLE productos (  
  id_producto VARCHAR(10) PRIMARY KEY,  
  nombre VARCHAR(100),  
  categoria VARCHAR(50),  
  marca VARCHAR(50),  
  precio_usd DECIMAL(10,2),  
  stock INTEGER,  
  color VARCHAR(50),  
  potencia_w INTEGER,  
  capacidad VARCHAR(20),  
  voltaje VARCHAR(20),  
  peso_kg DECIMAL(5,2),  
  garantia_meses INTEGER,  
  descripcion TEXT  
);
```

1.3. Manuales → Base de Datos de Grafos (Neo4j)

Objetivo: Extraer productos, componentes y sus relaciones de los manuales, y crear un grafo de conocimiento.

Pasos sugeridos:

1. Leer todos los archivos Markdown de `manuales_productos/`.
2. Usar un LLM para extraer entidades y relaciones.
3. Crear nodos y relaciones en Neo4j.

Nodos a crear:

- **Producto:** {id, nombre, categoria, marca}
- **Componente:** {nombre, tipo, especificacion}
- **Procedimiento:** {nombre, descripcion}

Relaciones a crear:

- (Producto)-[:TIENE_COMPONENTE]->(Componente)
- (Producto)-[:COMPATIBLE_CON]->(Producto)
- (Producto)-[:COMPARTE_REPUESTO {tipo}]->(Producto)
- (Componente)-[:USADO_EN]->(Producto)
- (Producto)-[:TIENE_PROCEDIMIENTO]->(Procedimiento)

Ejemplo de extracción:

- **Texto:** "El motor de 1200W es compartido con la Procesadora (P007)"

Extracción:

```
{
  "producto1": "P001",
  "producto2": "P007",
  "componente": "Motor 1200W",
  "relacion": "COMPARTE_COMPONENTE"
}
```

Consultas Cypher de ejemplo:

```
// Encontrar productos que comparten componentes
MATCH
(p1:Producto)-[:TIENE_COMPONENTE]->(c:Componente)-[:TIENE_COMPONENTE]->(p2:Producto)
RETURN p1.nombre, c.nombre, p2.nombre;

// Productos compatibles con P001
MATCH (p:Producto {id: 'P001'})-[:COMPATIBLE_CON]->(p2:Producto)
RETURN p2.nombre;
```

1.4. Reseñas y Manuales → Base de datos vectorial

Objetivo: Extraer vectores a partir de la información dada vectores y almacenar en una base de datos vectorial.

Pasos sugeridos:

1. Leer todos los archivos de texto de `resenas_usuarios/` y `manuales_productos/`.
2. Inicializar un divisor de texto en base a criterios propios (300 caracteres de longitud y 50 de overlap por ejemplo).

3. Fragmentar estos textos.
 4. Inicializar un modelo de embeddings que soporte español (por ejemplo `intfloat/multilingual-e5-small`).
 5. Vectorizar esos fragmentos de texto.
 6. Inicializar alguna base de datos vectorial (por ejemplo, ChromaDB).
 7. Almacenar esos fragmentos de texto vectorizados en la base de datos vectorial (**IMPORTANTE:** recordar almacenar Metadata a cada vector).
-

Parte 2: Interfaces de Bases de Datos

Crearán funciones que permitan consultar cada base de datos de manera programática.

2.1. Función de Consulta a Base de Datos

Objetivo: Crear una función para cada tipo de base de datos que ejecute consultas y devuelva resultados.

Funciones a implementar:

```
def consultar_resenas(filtro: dict) -> list:
    """
    Ejecuta una consulta en la colección de reseñas (MongoDB o JSON).

    Args:
        filtro: Diccionario con criterios de búsqueda
                Ej: {"producto_id": "P001", "puntaje": {"$gte": 4}}
    Returns:
        Lista de reseñas que cumplen el criterio
    """
    # Implementación
    pass

def consultar_productos(query: str) -> pd.DataFrame:
    """
    Ejecuta una consulta SQL o filtro de pandas.

    Args:
        query: Consulta SQL
                Ej: "SELECT * FROM productos WHERE precio_usd < 100"
    Returns:
        DataFrame con los resultados
    """
    # Implementación
    pass
```

```

def consultar_grafo(query_cypher: str) -> list:
    """
    Ejecuta una consulta Cypher en la base de grafos (Neo4j).

    Args:
        query_cypher: Consulta en lenguaje Cypher
            Ej: "MATCH (p:Producto {id:
'P001'})-[:TIENE_COMPONENTE]->(c) RETURN c.nombre"
    Returns:
        Lista de resultados de la consulta
    """
    # Implementación
    pass

def consultar_vectores(query: str, filtro: dict = {}, k: int) -> list:
    """
    Ejecuta una query en la base de datos vectorial.

    Args:
        query: Consulta a buscar
        filtro: Diccionario con criterios de búsqueda sobre la metadata
        k: número de vectores devueltos en la respuesta
    Returns:
        Lista de k vectores más cercanos
    """
    # Implementación
    pass

```

2.2. Función de Información de Base de Datos

Objetivo: Crear funciones que devuelvan metadatos sobre cada base de datos. Esta información se usará para que el LLM construya consultas correctas.

Funciones a implementar:

```

def info_resenas() -> dict:
    """
    Devuelve información sobre la estructura de las reseñas.
    """
    # Implementación
    # Ejemplo de retorno:
    # {
    #     "total_documentos": 85,
    #     "campos": ["id_resena", "fecha", "usuario", ...],

```

```

#     "productos_unicos": ["P001", "P002", ...],
#     "rango_puntajes": [1, 5],
#     "ejemplo": {...}
# }
pass

def info_productos() -> dict:
    """
    Devuelve información sobre la tabla de productos (SQL).
    """
    # Implementación
    # Ejemplo de retorno:
    # {
    #     "total_productos": 20,
    #     "columnas": ["id_producto", "nombre", "categoria", ...],
    #     "categorias": ["Cocina", "Climatizacion", ...],
    #     "marcas": ["TechHome", "BrewMaster", ...],
    #     "rango_precios": [24.99, 299.99],
    #     "ejemplo": {...}
    # }
    pass

def info_grafo() -> dict:
    """
    Devuelve información sobre el grafo de conocimiento (Neo4j).
    """
    # Implementación
    # Ejemplo de retorno:
    # {
    #     "total_nodos": 150,
    #     "tipos_nodos": ["Producto", "Componente", "Procedimiento"],
    #     "tipos_relaciones": ["TIENE_COMPONENTE", "COMPATIBLE_CON",
...],
    #     "ejemplo_producto": {...},
    #     "ejemplo_relacion": {...}
    # }
    pass

```

NOTA: En el caso de la base de datos vectorial, el LLM no debe generar ninguna consulta extra ya que directamente vectorizaremos la consulta del usuario y con ese vector realizaremos la búsqueda.

Parte 3: Generador de Consultas Dinámicas

Crearán un sistema que convierta lenguaje natural en consultas de base de datos usando un LLM.

3.1. Generador de Consultas con LLM

Objetivo: Implementar una función que tome una pregunta en lenguaje natural y genere la consulta apropiada para la base de datos correspondiente.

Función a implementar:

```
def generar_consulta(
    pregunta: str,
    tipo_bd: str,
    info_estructura: dict
) -> str:
    """
    Genera una consulta para una base de datos específica usando un LLM.

    Args:
        pregunta: Pregunta del usuario en lenguaje natural.
        tipo_bd: Tipo de base de datos ("mongodb", "sql", "cypher").
        info_estructura: Información de estructura (de Parte 2.2).

    Returns:
        Consulta generada en el lenguaje apropiado.
    """

    # 1. Construir prompt para el LLM
    prompt = f"""
    Eres un experto en bases de datos. Convierte la siguiente pregunta
    en una consulta {tipo_bd}.

    ESTRUCTURA DE LA BASE DE DATOS:
    {info_estructura}

    PREGUNTA DEL USUARIO:
    {pregunta}

    CONSULTA (solo el código, sin explicación):
    """

    # 2. Llamar al LLM (llamar_llm es una función hipotética que deben
    implementar)
    # consulta = llamar_llm(prompt)
    consulta = "Implementar llamada a LLM" # Placeholder

    return consulta
```

Ejemplos de conversión esperados:

Pregunta	Tipo BD	Consulta Generada
"Productos de menos de \$50"	SQL	<code>SELECT * FROM productos WHERE precio_usd < 50</code>
"Reseñas con 5 estrellas"	MongoDB	<code>{"puntaje": 5}</code>
"Productos compatibles con P001"	Cypher	<code>MATCH (p:Producto {id:'P001'})-[:COMPATIBLE_CON]->(p2) RETURN p2</code>
"Componentes de la licuadora"	Cypher	<code>MATCH (p:Producto {nombre:'Licuadora Roja'})-[:TIENE_COMPONENTE]->(c) RETURN c</code>

3.2. Flujo de Ejecución de Consulta

Objetivo: Crear un flujo completo que tome la pregunta del usuario, genere la consulta, la ejecute y devuelva resultados.

Función a implementar:

```
def flujo_consulta(pregunta: str, tipo_bd: str) -> dict:
    """
    Flujo completo: pregunta → consulta → ejecución → resultados
    """
    try:
        # 1. Obtener información de estructura
        info = {}
        if tipo_bd == "sql":
            info = info_productos()
        elif tipo_bd == "mongodb":
            info = info_resenas()
        elif tipo_bd == "cypher":
            info = info_grafo()
        elif tipo_bd == "vector":
            resultados = consultar_vectores(pregunta)
            return {
                "pregunta": pregunta,
```

```

        "consulta_generada": pregunta,
        "resultados": resultados,
        "error": None
    }

    # 2. Generar consulta con LLM
    consulta = generar_consulta(pregunta, tipo_bd, info)

    # 3. Ejecutar consulta
    resultados = None
    if tipo_bd == "sql":
        resultados = consultar_productos(consulta)
    elif tipo_bd == "mongodb":
        # ¡Cuidado! eval() es inseguro en producción. Usar
        # json.loads o ast.literal_eval
        resultados = consultar_resenas(eval(consulta))
    elif tipo_bd == "cypher":
        resultados = consultar_grafo(consulta)

    # 4. Devolver resultados
    return {
        "pregunta": pregunta,
        "consulta_generada": consulta,
        "resultados": resultados,
        "error": None
    }

except Exception as e:
    return {
        "pregunta": pregunta,
        "consulta_generada": None, # O la consulta que falló
        "resultados": None,
        "error": str(e)
    }

```

Casos de prueba sugeridos:

```

# Test 1: Consulta SQL
resultado = flujo_consulta(
    "Muéstrame productos de cocina de menos de $100",
    "sql"
)
print(resultado)

# Test 2: Consulta MongoDB

```

```

resultado = flujo_consulta(
    "Reseñas positivas de la Licuadora Roja",
    "mongodb"
)
print(resultado)

# Test 3: Consulta Cypher
resultado = flujo_consulta(
    "¿Qué productos comparten componentes con la Procesadora?",
    "cypher"
)
print(resultado)

# Test 4: Consulta a vectores
resultado = flujo_consulta(
    "¿Como puedo rallar el queso con el nuevo rallador?",
    "vector"
)
print(resultado)

resultado = flujo_consulta(
    "¿Quien dijo que hace waffles para los chicos?",
    "vector"
)
print(resultado)

```

Parte 4: RAG (Retrieval Augmented Generation)

En esta parte final crearán un sistema RAG completo que permita hacer preguntas en lenguaje natural y obtener respuestas contextualizadas.

4.1. Sistema RAG Completo

Objetivo: Crear un sistema que combine búsqueda en bases de datos + búsqueda vectorial + generación con LLM.

Arquitectura del sistema:

1. **Usuario pregunta:** (Ej: "¿Cómo amaso pan con la Batidora?")
2. **Clasificador (LLM):** ¿Qué tipo de consulta es? (SQL, MongoDB, Cypher, VectorDB)
3. **Router:** Dirige la pregunta al sistema correspondiente.
 - Información de producto → SQL
 - Opiniones/reseñas → MongoDB
 - Relaciones/compatibilidad → Cypher
 - Instrucciones/procedimientos → Vector DB
4. **Ejecutar consulta(s):** Se obtienen los datos (contexto).
5. **Generador (LLM):** Recopila los resultados y genera una respuesta en lenguaje

natural.

6. **Respuesta:** Se devuelve al usuario.

Funciones principales:

```
def clasificar_pregunta(pregunta: str) -> str:
    """
    Clasifica el tipo de pregunta para saber qué BD consultar.
    Returns: "producto" | "resenas" | "relaciones" | "procedimiento"
    """
    prompt = f"""
    Clasifica la siguiente pregunta en UNA de estas categorías:
    - "producto": Preguntas sobre características, precios, stock de
    productos.
    - "resenas": Preguntas sobre opiniones, experiencias,
    calificaciones.
    - "relaciones": Preguntas sobre compatibilidad, componentes
    compartidos, repuestos.
    - "procedimiento": Preguntas sobre cómo usar, instrucciones, pasos a
    seguir.
    - "resenas_detallado": Preguntas detalladas y puntuales sobre
    comentarios dentro de resenas.

    PREGUNTA: {pregunta}

    CATEGORIA (una palabra):
    """
    # categoria = Llamar_Llm(prompt).strip().Lower()
    categoria = "Implementar llamada a LLM" # Placeholder
    return categoria

def sistema_rag(pregunta: str) -> str:
    """
    Sistema RAG completo que responde preguntas sobre productos.
    """
    # 1. Clasificar tipo de pregunta
    tipo_consulta = clasificar_pregunta(pregunta)

    # 2. Ejecutar consulta(s) apropiada(s) y obtener contexto
    contexto = ""

    if tipo_consulta == "producto":
        resultado_flujo = flujo_consulta(pregunta, "sql")
        if resultado_flujo["resultados"] is not None:
```

```

        contexto += f"DATOS DE
PRODUCTOS:\n{resultado_flujo['resultados'].to_string()}\n\n"

    elif tipo_consulta == "resenas":
        resultado_flujo = flujo_consulta(pregunta, "mongodb")
        if resultado_flujo["resultados"] is not None:
            contexto += f"RESENAS:\n{resultado_flujo['resultados']}\n\n"

    elif tipo_consulta == "relaciones":
        resultado_flujo = flujo_consulta(pregunta, "cypher")
        if resultado_flujo["resultados"] is not None:
            contexto +=
f"RELACIONES:\n{resultado_flujo['resultados']}\n\n"

    elif tipo_consulta == "procedimiento":
        resultados = buscar_procedimientos(pregunta, k=2)
        for doc, score in resultados:
            contexto += f"PROCEDIMIENTO (Relevancia:
{score:.2f}):\n{doc.page_content}\n\n"

    elif tipo_consulta == "resenas_detallado":
        resultados = buscar_procedimientos(pregunta, k=2)
        for doc, score in resultados:
            contexto += f"RESEÑAS OBTENIDAS (Relevancia:
{score:.2f}):\n{doc.page_content}\n\n"

# 3. Generar respuesta con LLM
prompt = f"""
Eres un asistente virtual de una tienda de electrodomésticos.
Responde la pregunta del usuario usando la información
proporcionada.
Si la información no es suficiente, indica que no encontraste datos.

PREGUNTA DEL USUARIO:
{pregunta}

INFORMACIÓN DISPONIBLE (CONTEXTO):
{contexto}

RESPUESTA (en lenguaje natural, amigable y concisa):
"""

# respuesta = Llamar_Llm(prompt)
respuesta = "Implementar llamada a LLM" # Placeholder

return respuesta

```

Ejemplos de preguntas y respuestas esperadas:

Pregunta	Tipo	Bases consultadas	Respuesta esperada
"¿Cuál es el precio de la Licuadora Roja?"	producto	SQL	"La Licuadora Roja tiene un precio de \$89.99. Está disponible en color rojo con 45 unidades en stock."
"¿Qué opinan los usuarios de la Freidora de Aire?"	resenas	MongoDB	"La Freidora de Aire tiene excelentes opiniones con un promedio de 4.5/5 estrellas. Los usuarios destacan que cocina sin aceite y es fácil de usar."
"¿Qué productos son compatibles con la Procesadora?"	relaciones	Cypher	"La Procesadora (P007) es compatible con la Licuadora Roja (P001) ya que comparten el mismo motor de 1200W."
"¿Cómo amasar pan con la Batidora de Pie?"	procedimiento	Vector DB	"Para amasar pan con la Batidora de Pie: 1. Instala el gancho amasador, 2. Agrega ingredientes secos al bowl, 3. Inicia en velocidad 2..."

4.3. Conversación Continua (Bucle Interactivo)

Objetivo: Crear una interfaz de conversación donde el usuario puede hacer múltiples preguntas.

```
def chat_interactivo():  
    """  
    Interfaz de chat para consultar las bases de datos (RAG automático).  
    """  
    print("="*60)  
    print("ASISTENTE VIRTUAL - ELECTRODOMESTICOS (RAG Automático)")
```

```

print("="*60)
print("Escribe 'salir' o 'exit' para terminar.\n")

historial = []

while True:
    pregunta = input("TU PREGUNTA: ").strip()

    if not pregunta:
        continue
    if pregunta.lower() in ['salir', 'exit', 'quit']:
        print("\n¡Hasta luego!")
        break

    print("\nPensando...\n")
    try:
        respuesta = sistema_rag(pregunta) # Llama al RAG completo
        print(f"ASISTENTE: {respuesta}\n")
        print("-"*60 + "\n")
        historial.append({"pregunta": pregunta, "respuesta":
respuesta})
    except Exception as e:
        print(f"ERROR: {e}\nIntenta reformular la pregunta.\n")

def chat_con_seleccion_bd():
    """
    Chat donde el usuario selecciona qué BD consultar (para debugging).
    """
    print("="*60)
    print("ASISTENTE VIRTUAL - (Selección Manual de BD)")
    print("="*60)

    print("Bases de datos disponibles:")
    print("1. SQL (Productos)")
    print("2. MongoDB (Reseñas)")
    print("3. Neo4j (Relaciones)")
    print("4. Vectorial (Procedimientos)")

    bd_seleccionada = input("\n¿Qué base de datos quieres consultar?
(1-4): ").strip()
    mapeo_bd = {"1": "sql", "2": "mongodb", "3": "cypher", "4":
"vector"}
    tipo_bd = mapeo_bd.get(bd_seleccionada)

    if not tipo_bd:
        print("Opción inválida")

```

```

        return

    while True:
        pregunta = input(f"\nTU PREGUNTA [{tipo_bd.upper()}]: ").strip()
        if pregunta.lower() in ['salir', 'exit', 'cambiar']:
            break

        print("\nGenerando consulta y buscando...\n")
        if tipo_bd == "vector":
            resultados = buscar_procedimientos(pregunta, k=3)
            for i, (doc, score) in enumerate(resultados, 1):
                print(f"\n{i}. {doc.metadata['nombre']} (Score: {score:.3f})")
                print(f"    Producto: {doc.metadata['producto']}")
                print(f"    {doc.page_content[:200]}...\n")
            else:
                resultado = flujo_consulta(pregunta, tipo_bd)
                print(f"Consulta generada: {resultado['consulta_generada']}")
                print(f"\nResultados:\n{resultado['resultados']}\n")
                if resultado['error']:
                    print(f"ERROR: {resultado['error']}")
                print("-"*60)

if __name__ == "__main__":
    # Descomentar La versión que quieran usar:

    # Opción 1: Sistema RAG automático
    chat_interactivo()

    # Opción 2: Para probar cada BD por separado
    # chat_con_seleccion_bd()

```

Enlace al flujo final en formato MERMAID de la práctica 5

[Flujo de trabajo final de la práctica](#)